Got it — I'll break this **entire assignment into clear, neat, step-by-step instructions** so you can follow it easily and score full marks. Everything below is based on your question paper (page 1)

✅ FULL STEP-BY-STEP GUIDE

🧩 PART 1: Java Program Development (15 Marks)

Step 1: Create Project

1. Open **IntelliJ / Eclipse**
2. Create a **Maven Project**
3. Set:
    - Group ID: `com.fooddelivery`
    - Artifact ID: `order-system`

Step 2: Create Classes

Create these Java classes:

1. `Order.java`

```
public class Order {
    private int orderId;
    private String foodItem;
    private String status;

    public Order(int orderId, String foodItem) {
        this.orderId = orderId;
        this.foodItem = foodItem;
        this.status = "PLACED";
    }

    public int getOrderId() { return orderId; }
    public String getFoodItem() { return foodItem; }
    public String getStatus() { return status; }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

---

2. `OrderService.java`

```
import java.util.*;

public class OrderService {
    private Map<Integer, Order> orders = new HashMap<>();

    public String placeOrder(Order order) {
        if (orders.containsKey(order.getOrderId())) {
            return "Order already exists!";
        }
```

```
        orders.put(order.getOrderId(), order);
        return "Order placed successfully";
    }

    public String trackOrder(int orderId) {
        Order order = orders.get(orderId);
        if (order == null) {
            return "Order not found";
        }
        return order.getStatus();
    }
}
```

🧪 PART 2: JUnit Testing (15 Marks)

Step 3: Add JUnit Dependency (in `pom.xml`)

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.9.0</version>
    <scope>test</scope>
</dependency>
```

Step 4: Create Test Class

`OrderServiceTest.java`

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class OrderServiceTest {

    @Test
    public void testOrderSuccess() {
        OrderService service = new OrderService();
        Order order = new Order(1, "Pizza");

        String result = service.placeOrder(order);
        assertEquals("Order placed successfully", result);
    }

    @Test
    public void testDuplicateOrder() {
        OrderService service = new OrderService();
        Order order = new Order(1, "Burger");

        service.placeOrder(order);
        String result = service.placeOrder(order);

        assertEquals("Order already exists!", result);
    }

    @Test
    public void testTrackOrderFailure() {
        OrderService service = new OrderService();

        String result = service.trackOrder(99);
        assertEquals("Order not found", result);
    }
}
```

🔁 PART 3: CI/CD Pipeline (10 Marks)

Step 5: Push Project to GitHub

1. Initialize git:

```
git init
git add .
git commit -m "Initial commit"
```

2. Create repo on GitHub
3. Push:

```
git remote add origin <repo-url>
git push -u origin master
```

Step 6: GitHub Actions CI Pipeline

Create file:
`.github/workflows/ci.yml`

```
name: Java CI

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v2

    - name: Set up Java
      uses: actions/setup-java@v2
      with:
        java-version: '17'

    - name: Build with Maven
      run: mvn clean install
```

Step 7: Maven Build

Run:

```
mvn clean install
```

✔ This compiles + runs tests automatically

🐳 PART 4: Docker + Kubernetes (10 Marks)

Step 8: Create Dockerfile

```
FROM openjdk:17
COPY target/order-system.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 9: Build Docker Image

```
docker build -t yourusername/order-system .
```

Step 10: Push to Docker Hub

```
docker login
docker push yourusername/order-system
```

Step 11: Kubernetes Deployment YAML

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: order
  template:
    metadata:
      labels:
        app: order
    spec:
      containers:
      - name: order-container
        image: yourusername/order-system
        ports:
        - containerPort: 8080
```

Step 12: Kubernetes Service YAML

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  type: NodePort
  selector:
    app: order
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30007
```

Step 13: Deploy to Kubernetes

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

🎯 FINAL CHECKLIST (VERY IMPORTANT)

✅ Java classes created
✅ Order logic implemented
✅ JUnit tests written
✅ GitHub repo created
✅ CI pipeline working
✅ Maven build successful
✅ Docker image built & pushed
✅ Kubernetes deployed

💡 If you want next:

I can:

- Draw **architecture diagram**
- Give **viva answers**
- Help you **run this on your laptop step-by-step**
- Convert this into a **PDF submission file**

Just tell me 👍